

수업 04 · 2026-10-16

Git/GitHub

파일에서 바뀐 부분을 비교하고 Git으로 기록합니다. 로컬 저장소로 연습한 뒤 GitHub에서 코드를 검토하는 예를 살펴봅니다.

10:00~17:00 · 50분 수업 후 10분 휴식 · 중식 11:50~13:00

오늘 필수 활동 10개 · 교시별 설명과 예제를 보고 직접 해 봅니다.

Git/GitHub · 변경을 비교하고 기록하기

2026-10-16 · Git/GitHub

연습용 README를 고치며 파일 저장과 Git 기록의 차이를 배웁니다. status로 상태를 보고 diff로 바뀐 줄을 확인한 뒤 add와 commit으로 기록합니다. 로컬 공유 저장소로 보내기·받기를 확인하고 GitHub 리뷰는 계정 없는 제공 자료로 연습합니다.

수업은 50분씩 6교시입니다. 원본을 보존하고 work의 지정한 부분을 수정합니다. 계정이나 실제 장비 없이 수업용 예제 자료를 사용합니다.

지난 수업에서 오늘 수업으로

10/8에는 기록의 의미와 파일 표현을 읽었습니다. 오늘은 연습용 README를 고치며 파일 저장과 Git 기록의 차이를 배웁니다. status로 상태를 보고 diff로 바뀐 줄을 확인한 뒤 add와 commit으로 기록합니다. 로컬 공유 저장소로 보내기·받기를 확인하고 GitHub 리뷰는 계정 없는 제공 자료로 연습합니다.

먼저 구별할 화면

- 파일 탐색기는 저장한 파일의 위치를 찾습니다.
- 텍스트 편집기는 코드와 데이터를 읽고 고칩니다.
- 터미널은 명령을 입력하고 실제 실행 결과를 확인합니다.
- 웹 교안은 설명·원문·출력 예를 제공합니다. Git 명령을 실행하는 터미널은 아닙니다.

참여 순서

1. 압축을 일반 폴더에 풀고 data·code·work의 위치를 유지합니다.
2. lesson의 개념을 읽고 activity-guide의 해당 활동 입력을 엽니다.
3. work의 답칸에 예상과 이유를 먼저 쓰고 지정한 값을 수정합니다.
4. 저장 후 정확한 work 파일을 실행하거나 강사 출력·제공 예와 수동 비교합니다.
5. 확인 방식을 기록합니다. 개인 실행을 하지 않았으면 실행 성공으로 적지 않습니다.

도구를 준비하지 못했을 때

설치 문제로 수업을 멈추지 말고 제공한 전체 코드·출력과 동일 입력을 사용해 줄별 값을 직접 추적합니다. 종이에도 같은 답칸을 작성합니다. 개인 실행은 강사가 환경을 준비한 뒤 확인합니다.

[교안](#) · [실습](#) · [용어](#)

수업 전, 이 세 가지부터 확인하세요

앞선 내용이 기억나지 않으면 아래 질문부터 읽어 보세요. 먼저 답을 적고 풀이와 비교하면 됩니다.

저장과 기록

질문: 파일을 저장하면 Git 이력도 자동으로 생기나요?
풀이: 저장은 작업 파일을 바꿉니다. 변경 확인, add, commit을 거쳐 기록을 남기는 절차는 별개입니다.

내 컴퓨터와 공유 위치

질문: 로컬 commit 후 다른 사람도 바로 그 내용을 받나요?
풀이: 원격 저장소에 보내는 과정은 별도입니다. 로컬 기록과 원격 반영을 구별해 확인합니다.

충돌

질문: 두 사람이 같은 줄을 바꾸면 최신 파일로 덮으면 될까요?
풀이: 두 변경의 의도를 비교해 결과를 결정하고 검증해야 합니다. 단순 덮어쓰기는 필요한 변경을 잃을 수 있습니다.

오늘의 수업과 복습을 나누는 방법

수업은 10:0010:50, 11:0011:50, 13:0013:50, 14:0014:50, 15:0015:50, 16:0016:50입니다. 중식은 11:5013:00이며 수업 사이에는 10분 쉽니다. 마지막 16:5017:00은 휴식과 저장·퇴실 준비로 배치했습니다.

교시	수업에서 먼저 확인할 활동	추가·복습
1	A01, A02, A03	새 입력으로 한 번 더 확인
2	A04, A05, A06	새 입력으로 한 번 더 확인
3	A07, A08, A09	새 입력으로 한 번 더 확인
4	A10, A11	A12
5	A13, A15	A14
6	A16, A17, A18	새 입력으로 한 번 더 확인

교안의 사례와 풀이를 읽고, 이번 교시에 정한 활동을 직접 해 봅니다. 활동 결과를 확인한 뒤에는 추가 문제를 풀어 봅니다. 모든 페이지를 같은 시간에 끝내라는 뜻은 아닙니다. 설치가 막혔을 때는 제공 출력과 수동 추적으로 개념을 확인하고, 직접 실행했는지는 따로 적습니다.

2026-10-16 · Git/GitHub

연습용 README를 고치며 파일 저장과 Git 기록의 차이를 배웁니다. status로 상태를 보고 diff로 바뀐 줄을 확인한 뒤 add와 commit으로 기록합니다. 로컬 공유 저장소로 보내기·받기를 확인하고 GitHub 리뷰는 계정 없는 제공 자료로 연습합니다.

수업은 50분 × 6교시, 순수 수업 300분입니다. 사례와 풀이를 읽으며 개념을 익히고, 새 문제는 복습으로 풉니다. 교시별로 먼저 할 활동은 시작 안내를 확인하세요. 학생이 끝내는 데 걸리는 시간은 아직 측정하지 않았습니다. 원본을 보존하고 work의 지정한 부분을 수정합니다. 계정이나 실제 장비 없이 수업용 예제 자료를 사용합니다.

1교시 · 작업 폴더와 저장소

10:00~10:50 · 설명15분 / 시연8분 / 직접 수행20분 / 풀이7분

A02 · 연습 저장소와 로컬 작성자 설정

개념. git init은 지정한 폴더를 Git 저장소로 시작합니다. .git에는 변경 기록과 설정이 들어 있습니다. 작성자 이름과 이메일은 커밋에 남는 정보이며 로그인 암호가 아닙니다.

왜 필요한가. 실제 프로젝트와 분리한 연습 폴더를 사용해야 명령의 범위를 분명히 볼 수 있습니다. 설정은 --local로 이번 저장소에만 적용하며 컴퓨터 전체 설정을 바꾸지 않습니다.

함께 보는 예. course의 work 아래 git-lab을 만들고 그 안에서 시작합니다. main은 첫 브랜치 이름입니다. user.email은 실습용 .invalid 주소를 쓰며 실제 GitHub 계정 인증을 수행하지 않습니다.

헛갈리는 경우. 상위 KDigital_PhysicalAI 프로젝트에서 init-restore를 실행하지 않습니다. 매번 현재 위치와 git rev-parse 결과로 연습 저장소 경계를 확인합니다.

전체 명령

```
New-Item -ItemType Directory -Path work/git-lab
Set-Location work/git-lab
git init -b main
git config --local user.name "실습자"
git config --local user.email "learner@example.invalid"
git rev-parse --show-toplevel
```

각 줄의 뜻

줄	코드	뜻
1	New-Item -ItemType Directory -Path work/git-lab	연습 전용 폴더를 만듭니다.
2	Set-Location work/git-lab	그 폴더로 현재 위치를 옮깁니다.
3	git init -b main	main으로 시작하는 로컬 저장소를 만듭니다.
4	git config --local user.name "실습자"	이번 저장소의 작성자 이름을 정합니다.
5	git config --local user.email "learner@example.invalid"	실습용 작성자 주소를 정합니다.
6	git rev-parse --show-toplevel	저장소 최상위 주소가 git-lab인지 확인합니다.

실행 위치와 확인

처음 위치는 압축을 푼 course 폴더입니다. Windows PowerShell 예시이며 다른 OS의 폴더 생성·이동은 강사 안내를 따릅니다. git-lab이 이미 있으면 새 복사본의 빈 폴더를 사용합니다.

대표 확인 결과(커밋 해시·사용자명 등은 환경마다 다름):

```

Initialized empty Git repository in .../work/git-lab/.git/
.../work/git-lab
    
```

학생 적용

1. 일반 폴더로 압축을 풉니다.
2. PowerShell의 현재 위치가 course인지 확인합니다.
3. 명령을 한 줄씩 실행하며 작성자 설정의 범위를 설명합니다.
4. 최상위 주소 끝이 work/git-lab인지 기록합니다.

예상 결과: 연습 폴더 안의 .git 생성과 로컬 설정. 아직 커밋은 없습니다.

확인 질문: 이메일 설정은 GitHub 로그인인가요?

막혔을 때: git 명령이 없으면 강사가 설치 상태를 확인합니다. 제공한 명령·상태 카드로 수동 추적하고 개인 실행은 미완료로 표시합니다.

[활동 자료와 작성칸](#)에서 A02를 엽니다.

A02 · 직접 해보기

A02 · 연습 저장소와 로컬 작성자 설정

목적: 실제 프로젝트와 분리한 연습 폴더를 사용해야 명령의 범위를 분명히 볼 수 있습니다. 설정은 --local로 이번 저장소에만 적용하며 컴퓨터 전체 설정을 바꾸지 않습니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A02_input.md](#)
- [A02_setup.txt](#)
- [A02_notes.md](#)
- [A02_setup.txt](#)

단계

1. 일반 폴더로 압축을 풉니다.
2. PowerShell의 현재 위치가 course인지 확인합니다.
3. 명령을 한 줄씩 실행하며 작성자 설정의 범위를 설명합니다.

4. 최상위 주소 끝이 work/git-lab인지 기록합니다.
- 예상 결과:** 연습 폴더 안의 .git 생성과 로컬 설정. 아직 커밋은 없습니다.
- 확인 방법:** 이메일 설정은 GitHub 로그인인가요?

오류 해결: git 명령이 없으면 강사가 설치 상태를 확인합니다. 제공한 명령·상태 카드로 수동 추적하고 개인 실행은 미완료로 표시합니다.

수정 전 예상: _____

확인한 결과와 이유: _____

A03 · status로 현재 상태 읽기

개념. git status는 현재 작업 파일과 스테이징 상태를 보여 줍니다. 처음 만든 파일의 ??는 아직 Git이 추적하지 않는 파일이라는 짧은 표시입니다.

왜 필요한가. 파일이 폴더에 존재한다는 사실과 이력에 들어갔다는 사실을 구별해야 합니다. 명령을 실행한 뒤 상태를 읽으면 어느 단계까지 했는지 확인할 수 있습니다.

함께 보는 예. data/initial_README.md의 내용을 탐색기로 git-lab/README.md에 복사하면 아직 추적하지 않은 파일로 나타납니다. status만 보는 것은 기록을 만들거나 파일을 바꾸지 않습니다.

헛갈리는 경우. working tree clean이라는 문구는 현재 비교할 변경이 없다는 뜻입니다. 코드가 기능적으로 올바르거나 GitHub에 올라갔다는 뜻은 아닙니다.

전체 명령

```
git status
git status --short
```

각 줄의 뜻

줄	코드	뜻
1	git status	자세한 현재 상태를 읽습니다.
2	git status --short	같은 상태를 짧은 기호로 확인합니다.

실행 위치와 확인

현재 위치: work/git-lab. 먼저 원자료 initial_README.md를 README.md로 복사하여 저장합니다.

대표 확인 결과(커밋 해시·사용자명 등은 환경마다 다름):

```
?? README.md
```

학생 적용

1. initial_README.md를 새 README.md로 복사합니다.

2. 상태 명령 두 가지를 비교합니다.
3. ??의 의미와 아직 하지 않은 행동을 씁니다.
4. 파일 내용과 변경 상태가 별개임을 설명합니다.

예상 결과: README가 untracked 또는??로 표시됩니다.

확인 질문: status를 열 번 실행하면 커밋이 생기나요?

막혔을 때: README가 안 보이면 저장한 경로와 파일 확장자를 확인합니다. 다른 폴더의 파일을 보고 있을 수 있습니다.

[활동 자료와 작성칸](#)에서 A03를 엽니다.

A03 · 직접 해보기

A03 · status로 현재 상태 읽기

추가 원문·작업 파일:

- [A03_input.md](#)
- [A03_status.txt](#)
- [initial_README.md](#)
- [A03_notes.md](#)
- [A03_status.txt](#)

목적: 파일이 폴더에 존재한다는 사실과 이력에 들어갔다는 사실을 구별해야 합니다. 명령을 실행한 뒤 상태를 읽으면 어느 단계까지 했는지 확인할 수 있습니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A03_input.md](#)
- [A03_status.txt](#)
- [A03_notes.md](#)
- [A03_status.txt](#)

단계

1. initial_README.md를 새 README.md로 복사합니다.
2. 상태 명령 두 가지를 비교합니다.
3. ??의 의미와 아직 하지 않은 행동을 씁니다.

4. 파일 내용과 변경 상태가 별개임을 설명합니다.

예상 결과: README가 untracked 또는??로 표시됩니다.

확인 방법: status를 열 번 실행하면 커밋이 생기나요?

오류 해결: README가 안 보이면 저장한 경로와 파일 확장자를 확인합니다. 다른 폴더의 파일을 보고 있을 수 있습니다.

수정 전 예상: _____

확인한 결과와 이유: _____

함께 보는 예와 더 해보기: 파일 저장과 변경 기록

[이 교시의 다른 예와 코드 해설 열기](#)

2교시 · 선택한 변경을 기록하기

11:00~11:50 · 설명15분 / 시연8분 / 직접 수행20분 / 풀이7분

A05 · add는 현재 변경을 선택한다

개념. git add는 지금의 파일 내용을 다음 커밋에 넣을 상태로 선택합니다. add 뒤 파일을 다시 고쳐도 스테이징이 자동으로 최신 내용으로 바뀌지는 않습니다.

왜 필요한가. 기록할 묶음을 고르고 검토해야 의도한 변경만 커밋할 수 있습니다. 'add 했으니 이후 모든 저장도 기록된다'는 오해를 막습니다.

함께 보는 예. 처음 README를 add하면 짧은 상태의 왼쪽 A가 추가 준비를 나타냅니다. --staged는 선택한 상태와 마지막 커밋을 비교합니다. 첫 커밋 전에는 선택한 전체 내용을 볼 수 있습니다.

헷갈리는 경우. git add .는 여러 파일을 한꺼번에 고를 수 있습니다. 초반에는 README.md처럼 이름을 명시해 무엇을 선택했는지 이해합니다.

전체 명령

```
git add README.md
git status --short
git diff --staged
```

각 줄의 뜻

줄	코드	뜻
1	git add README.md	README의 현재 상태를 선택합니다.
2	git status --short	선택 상태를 짧게 읽습니다.
3	git diff --staged	이번 기록에 들어갈 내용을 비교합니다.

실행 위치와 확인

현재 위치: work/git-lab. README가 저장되어 있고 첫 커밋 전 상태입니다.

대표 확인 결과(커밋 해시·사용자명 등은 환경마다 다름):

```
A  README.md
선택한 README 전체 내용이 추가 줄로 보입니다.
```

학생 적용

1. README 한 파일만 add합니다.
2. 상태 왼쪽 표시와 staged 차이를 읽습니다.
3. 지금 커밋하면 들어갈 문장을 적습니다.
4. add 뒤 편집한 가상 카드에서 스테이징과 작업 영역을 나눕니다.

예상 결과: 선택한 파일은A로 표시되고 staged 비교에 내용이 나타납니다.

확인 질문: add 뒤 고친 내용까지 넣으려면 무엇을 확인해야 하나요?

막혔을 때: staged에 예상 밖 내용이 있으면 커밋 전에 파일 경로와 비교를 확인합니다.

[활동 자료와 작성칸](#)에서 A05를 엽니다.

A05 · 직접 해보기

A05 · add는 현재 변경을 선택한다

목적: 기록할 묶음을 고르고 검토해야 의도한 변경만 커밋할 수 있습니다. ‘add 했으니 이후 모든 저장도 기록된다’는 오해를 막습니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A05_input.md](#)

- [A05_stage.txt](#)

- [A05_notes.md](#)

- [A05_stage.txt](#)

단계

1. README 한 파일만 add합니다.
2. 상태 왼쪽 표시와 staged 차이를 읽습니다.
3. 지금 커밋하면 들어갈 문장을 적습니다.

4. add 뒤 편집한 가상 카드에서 스테이징과 작업 영역을 나눕니다.

예상 결과: 선택한 파일은A로 표시되고 staged 비교에 내용이 나타납니다.

확인 방법: add 뒤 고친 내용까지 넣으려면 무엇을 확인해야 하나요?

오류 해결: staged에 예상 밖 내용이 있으면 커밋 전에 파일 경로와 비교를 확인합니다.

수정 전 예상: _____

확인한 결과와 이유: _____

A06 · commit과 변경 이력

개념. 커밋은 스테이징한 상태를 메시지와 함께 이력에 남깁니다. 커밋에는 식별 해시와 작성자·메시지 등이 있으며 해시 값은 각 실습 환경에서 달라질 수 있습니다.

왜 필요한가. 무엇을 왜 바꿨는지 나중에 확인할 수 있도록 기록합니다. 파일 저장→선택→기록의 세 단계를 직접 수행합니다.

함께 보는 예. 첫 README를 'Add checker guide'라는 메시지로 기록합니다. 로그에서 짧은 해시와 메시지를 찾고 status에서 남은 변경을 확인합니다.

헛갈리는 경우. commit은 GitHub로 보내는 명령이 아닙니다. 로컬 기록이 생겨도 공유 저장소에 전송하려면 별도 작업이 필요합니다.

전체 명령

```
git commit -m "Add checker guide"
git log --oneline
git status --short
```

각 줄의 뜻

줄	코드	뜻
1	git commit -m "Add checker guide"	선택한 상태를 메시지와 함께 기록합니다.
2	git log --oneline	짧은 커밋 이력을 봅니다.
3	git status --short	기록 뒤 남은 변경이 있는지 확인합니다.

실행 위치와 확인

현재 위치: work/git-lab. A05에서 README를 스테이징했습니다.

대표 확인 결과(커밋 해시·사용자명 등은 환경마다 다름):

```
<짧은해시> Add checker guide
status --short는 남은 변경이 없으면 빈 출력입니다.
```

학생 적용

1. 선택한 내용을 마지막으로 확인합니다.
2. 첫 커밋을 만듭니다.
3. 자신의 실제 해시와 메시지를 기록합니다.
4. 아직 수행하지 않은 공유 단계를 표시합니다.

예상 결과: 첫 커밋 한 개. 남은 변경이 없으면 상태가 깨끗합니다.

확인 질문: 커밋 해시가 교안과 달라지면 실패인가요?

막혔을 때: 작성자 오류는 로컬 user.name과user.email 설정을 확인합니다. 글로벌 설정을 임의로 바꾸지 않습니다.

활동 자료와 작성칸에서 A06를 엽니다.

A06 · 직접 해보기

A06 · commit과 변경 이력

목적: 무엇을 왜 바꿨는지 나중에 확인할 수 있도록 기록합니다. 파일 저장→선택→기록의 세 단계를 직접 수행합니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A06_input.md](#)
- [A06_commit.txt](#)
- [A06_notes.md](#)
- [A06_commit.txt](#)

단계

1. 선택한 내용을 마지막으로 확인합니다.
2. 첫 커밋을 만듭니다.
3. 자신의 실제 해시와 메시지를 기록합니다.
4. 아직 수행하지 않은 공유 단계를 표시합니다.

예상 결과: 첫 커밋 한 개. 남은 변경이 없으면 상태가 깨끗합니다.

확인 방법: 커밋 해시가 교안과 달라지면 실패인가요?

오류 해결: 작성자 오류는 로컬 user.name과user.email 설정을 확인합니다. 글로벌 설정을 임의로 바꾸지 않습니다.

수정 전 예상: _____

확인한 결과와 이유: _____

함께 보는 예와 더 해보기: 작업 변경과 diff의 기준

[이 교사의 다른 예와 코드 해설 열기](#)

3교시 · 기록의 의미와 관리 대상

13:00~13:50 · 설명15분 / 시연8분 / 직접 수행20분 / 풀이7분

A09 · .gitignore와 관리 대상

개념. .gitignore는 특정 경로의 추적되지 않은 파일을 상태와 추가 대상에서 무시하도록 정하는 규칙입니다. 이미 추적한 파일의 과거 기록을 삭제하는 기능은 아닙니다.

왜 필요한가. 임시 실행 결과나 캐시를 코드·안내 변경과 섞지 않도록 관리 범위를 정합니다. 무시 규칙이 무엇을 포함하는지 직접 확인해야 합니다.

함께 보는 예. scratch.txt를 무시 목록에 적고 임시 메모를 만듭니다. status에서 그 파일이 빠지고 check-ignore로 적용된 규칙을 볼 수 있습니다. .gitignore 자체는 공유할 설정으로 커밋할 수 있습니다.

헛갈리는 경우. 이미 커밋한 파일 이름을 .gitignore에 넣어도 이력에서 사라지지 않습니다. 공개 범위를 숨기기 위한 도구로 오해하지 않습니다.

전체 명령

```
git status --short
git check-ignore -v scratch.txt
git add .gitignore
git commit -m "Ignore scratch notes"
```

각 줄의 뜻

줄	코드	뜻
1	<code>git status --short</code>	어떤 파일이 변경 대상으로 보이는지 확인합니다.
2	<code>git check-ignore -v scratch.txt</code>	임시 파일을 무시한 규칙을 찾습니다.
3	<code>git add .gitignore</code>	무시 규칙 파일만 선택합니다.
4	<code>git commit -m "Ignore scratch notes"</code>	관리 대상 규칙을 기록합니다.

실행 위치와 확인

work/git-lab에 편집기로 .gitignore를 만들고 한 줄 scratch.txt를 씁니다. 임시 scratch.txt도 만듭니다.

대표 확인 결과(커밋 해시·사용자명 등은 환경마다 다름):

```
.gitignore:1:scratch.txt scratch.txt
scratch.txt는 일반 상태 목록에서 제외됩니다.
```

학생 적용

1. 무시할 임시 파일과 보존할 안내 파일을 고릅니다.
2. 규칙과 임시 파일을 만듭니다.
3. status와 check-ignore를 비교합니다.
4. 이미 추적된 파일과의 차이를 설명합니다.

예상 결과: scratch는 무시되고 .gitignore 규칙이 기록됩니다.

확인 질문: .gitignore가 과거 커밋의 파일을 지우나요?

막혔을 때: .gitignore.txt로 저장되었는지 확인합니다. 정확한 파일명과 규칙 경로를 봅니다.

[활동 자료와 작성칸](#)에서 A09를 엽니다.

A09 · 직접 해보기

A09 · .gitignore와 관리 대상

목적: 임시 실행 결과나 캐시를 코드·안내 변경과 섞지 않도록 관리 범위를 정합니다. 무시 규칙이 무엇을 포함하는지 직접 확인해야 합니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A09_input.md](#)

- [A09_ignore.txt](#)

- [A09_notes.md](#)

- [A09_ignore.txt](#)

단계

1. 무시할 임시 파일과 보존할 안내 파일을 고릅니다.
2. 규칙과 임시 파일을 만듭니다.
3. status와 check-ignore를 비교합니다.
4. 이미 추적된 파일과의 차이를 설명합니다.

예상 결과: scratch는 무시되고 .gitignore 규칙이 기록됩니다.

확인 방법: .gitignore가 과거 커밋의 파일을 지우나요?

오류 해결: .gitignore.txt로 저장되었는지 확인합니다. 정확한 파일명과 규칙 경로를 봅니다.

수정 전 예상: _____

확인한 결과와 이유: _____

함께 보는 예와 더 해보기: 한 목적의 변경과 메시지 / 현재 내용과 과거 커밋 비교

이 교시의 다른 예와 코드 해설 열기

4교시 · 비교와 범위 있는 복구

14:00~14:50 · 설명15분 / 시연8분 / 직접 수행20분 / 풀이7분

A10 · 스테이징 취소와 작업 내용 유지

사례로 이해하기 · 취소하려는 대상부터 정하는 복구

실무 상황

동료가 README를 잘 고쳤지만 아직 검토하지 않아 이번 커밋에서 빼고 싶습니다. 다른 경우에는 연습 파일에 쓴 틀린 문장 자체를 버리고 마지막 기록으로 돌아가고 싶습니다. 둘은 취소 대상이 다르므로 같은 명령을 쓰면 필요한 내용을 잃을 수 있습니다.

쉬운 풀이

스테이징 취소는 제출 봉투에서 사본을 빼는 일입니다. 작업 내용 복구는 책상 위 종이를 이전 사본으로 바꾸는 일입니다. `git restore --staged README.md`는 기본적으로 마지막 커밋을 기준으로 선택 상태를 돌립니다. 작업 파일의 문장은 유지합니다. `git restore --source=HEAD --worktree README.md`는 작업 파일을 마지막 커밋 내용으로 바꿉니다. 기록하지 않은 편집 내용은 사라질 수 있으므로 복사본을 남기고 대상 파일을 명시합니다.

완성 예와 단계 풀이

마지막 기록 문장이 `Run from course`.인 연습 저장소에서 README를 `Run from wrong folder`.로 바꾸고 add합니다. 먼저 staged diff를 읽고 `git restore --staged README.md`를 실행합니다. status에는 아직 작업 변경이 남습니다. 탐색기나 편집기로 문장도 확인합니다. 잘못된 문장을 버리는 것이 확정되었을 때만 `git restore --source=HEAD --worktree README.md`를 실행합니다. 문장이 다시 course로 돌아오고 작업 차이도 사라집니다. 다른 파일에는 이 명령을 확장하지 않습니다.

독립 추가 복습 과제

“이번 제출에서만 빼기”와 “틀린 문장 폐기하기”에 각각 맞는 명령을 고르세요. 실행 전에 현재 파일 내용과 스테이징 상태를 적어 두세요. 아직 커밋하지 않은 새 파일도 같은 방식으로 과거 내용에 복구할 수 있는지 판단하세요.

답과 대표 오답

제출 선택만 취소할 때는 staged 옵션을, 작업 내용을 이전 기록으로 바꿀 때는 source와 worktree를 선택합니다. 새 파일에 과거 기록이 없으면 돌아갈 사본도 없습니다. “restore는 언제나 파일을 안전하게 보관한다”는 오답입니다. 목적과 기준을 읽지 않은 복구는 편집 내용을 버릴 수 있습니다.

확인 질문과 해설

① 취소 뒤 문장이 남는 것은 실패인가요? 선택만 취소했다면 정상입니다. ② HEAD는 무엇인가요? 현재 마지막 커밋을 가리킵니다. ③ 복구 확인은 무엇으로 하나요? status만 보지 말고 파일 내용과 비교 결과를 함께 확인합니다.

개념. `git restore --staged`는 선택한 파일의 스테이징 상태를 현재 커밋 기준으로 되돌립니다. 작업 파일의 편집 내용은 유지합니다. 기본 restore와 대상이 다릅니다.

왜 필요한가. 실수로 선택한 변경을 커밋 전에 빼돌려 작성한 내용은 계속 보관하고 싶은 경우가 있습니다. 취소 대상이 파일 내용인지 다음 기록의 선택인지 구별해야 합니다.

함께 보는 예. README에 Practice note.를 추가해 add한 뒤 --staged로 선택을 취소합니다. staged 비교는 비교 작업 diff에는 문장이 남습니다.

헛갈리는 경우. git restore README.md는 작업 파일 내용을 바꾸는 명령입니다. --staged가 없는 명령과 같은 결과라고 가정하지 않습니다.

전체 명령

```
git add README.md
git diff --staged
git restore --staged README.md
git diff --staged
git diff -- README.md
```

각 줄의 뜻

줄	코드	뜻
1	git add README.md	현재 변경을 선택합니다.
2	git diff --staged	선택 내용을 확인합니다.
3	git restore --staged README.md	다음 커밋의 선택만 취소합니다.
4	git diff --staged	선택 취소 결과를 확인합니다.
5	git diff -- README.md	작업 파일에 남은 편집을 확인합니다.

실행 위치와 확인

work/git-lab의 README 끝에 Practice note.를 추가해 저장합니다. 현재 내용을 별도 노트에도 기록해 둡니다.

대표 확인 결과(커밋 해시·사용자명 등은 환경마다 다름):

```
staged 차이는 사라집니다.
작업 diff에는 +Practice note.가 남습니다.
```

학생 적용

1. 취소 전에 작업과 staged 상태를 그립니다.
2. README 한 파일만 선택 취소합니다.
3. 파일을 직접 열어 문구가 유지되는지 확인합니다.

4. 선택 취소와 내용 복구를 문장으로 구별합니다.

예상 결과: 작업 문구 유지, staged 변경 없음.

확인 질문: 스테이징을 취소하면 편집한 문구가 사라지나요?

막혔을 때: 복구 명령을 외워 입력하기 전에 옵션과 대상 파일을 읽고 현재 상태를 기록합니다.

[활동 자료와 작성칸](#)에서 A10를 엽니다.

A10 · 직접 해보기

A10 · 스테이징 취소와 작업 내용 유지

목적: 실수로 선택한 변경을 커밋 전에 빠뜨린 작성한 내용은 계속 보관하고 싶은 경우가 있습니다. 취소 대상이 파일 내용인지 다음 기록의 선택인지 구별해야 합니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A10_input.md](#)

- [A10_unstage.txt](#)

- [A10_notes.md](#)

- [A10_unstage.txt](#)

단계

1. 취소 전에 작업과 staged 상태를 그립니다.
2. README 한 파일만 선택 취소합니다.
3. 파일을 직접 열어 문구가 유지되는지 확인합니다.
4. 선택 취소와 내용 복구를 문장으로 구별합니다.

예상 결과: 작업 문구 유지, staged 변경 없음.

확인 방법: 스테이징을 취소하면 편집한 문구가 사라지나요?

오류 해결: 복구 명령을 외워 입력하기 전에 옵션과 대상 파일을 읽고 현재 상태를 기록합니다.

수정 전 예상: _____

확인한 결과와 이유: _____

A11 · 연습 파일 한 개의 작업 내용 복구

개념. git restore -- README.md는 기본적으로 스테이징 상태를 기준으로 작업 파일을 복구합니다. A10에서 선택을 취소한 현재 상황에서는 최근 커밋의 README 내용으로 돌아갑니다.

왜 필요한가. 연습 변경을 버릴 때 어느 내용을 잃고 어느 기록이 남는지 알고 수행해야 합니다. 전체 폴더가 아닌 한 파일로 범위를 한정하고 현재 내용을 따로 보관합니다.

함께 보는 예. Practice note.를 백업 텍스트에 복사한 뒤 README만 복구합니다. 문구는 작업 파일에서 사라지지만 이미 만든 커밋 이력은 유지됩니다.

햇갈리는 경우. 복구 전에 따로 기록하지 않은 미커밋 작업은 Git 이력으로 되찾을 수 있다고 보장하지 않습니다. 모든 폴더를 강제로 되돌리는 명령으로 확장하지 않습니다.

전체 명령

```
git diff -- README.md
git restore -- README.md
git status --short
git log --oneline -3
```

각 줄의 뜻

줄	코드	뜻
1	<code>git diff -- README.md</code>	복구로 없어질 작업 차이를 확인합니다.
2	<code>git restore -- README.md</code>	지정한 README 작업 내용만 복구합니다.
3	<code>git status --short</code>	남은 변경을 확인합니다.
4	<code>git log --oneline -3</code>	기존 커밋 이력이 유지되는지 봅니다.

실행 위치와 확인

work/git-lab이며 A10을 마친 상태입니다. Practice note.가 들어 있는 현재 README를 work/README_before_restore.txt로 복사해 보관한 뒤 진행합니다.

대표 확인 결과(커밋 해시·사용자명 등은 환경마다 다름):

```
Practice note. 작업 변경이 사라집니다.
기존 커밋은 그대로 유지됩니다.
```

학생 적용

1. 사라질 문구와 백업 위치를 먼저 씁니다.
2. 연습 저장소 주소를 다시 확인합니다.
3. 한 파일만 복구하고 직접 엽니다.
4. 기록 이력과 작업 내용의 차이를 설명합니다.

예상 결과: README에서 임시 문구가 없어지고 과거 커밋은 유지됩니다.

확인 질문: restore는 새 커밋을 만드는데요?

막혔을 때: 주소가 실제 프로젝트라면 이 연습을 수행하지 말고 별도 git-lab로 이동합니다. 백업과 대상 파일명을 먼저 확인합니다.

[활동 자료와 작성칸](#)에서 A11를 엽니다.

A11 · 직접 해보기

A11 · 연습 파일 한 개의 작업 내용 복구

목적: 연습 변경을 버릴 때 어느 내용을 잃고 어느 기록이 남는지 알고 수행해야 합니다. 전체 폴더가 아닌 한 파일로 범위를 한정하고 현재 내용을 따로 보관합니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A11_input.md](#)

- [A11_restore.txt](#)

- [A11_notes.md](#)

- [A11_restore.txt](#)

단계

1. 사라질 문구와 백업 위치를 먼저 씁니다.
2. 연습 저장소 주소를 다시 확인합니다.
3. 한 파일만 복구하고 직접 엽니다.
4. 기록 이력과 작업 내용의 차이를 설명합니다.

예상 결과: README에서 임시 문구가 없어지고 과거 커밋은 유지됩니다.

확인 방법: restore는 새 커밋을 만드는데요?

오류 해결: 주소가 실제 프로젝트라면 이 연습을 수행하지 말고 별도 git-lab로 이동합니다. 백업과 대상 파일명을 먼저 확인합니다.

수정 전 예상: _____

확인한 결과와 이유: _____

함께 보는 예와 더 해보기: 브랜치와 병합의 흐름

[이 교시의 다른 예와 코드 해설 열기](#)

5교시 · 공유 저장소와 리뷰

15:00~15:50 · 설명15분 / 시연8분 / 직접 수행20분 / 풀이7분

A15 · GitHub와 PR의 역할

개념. Git은 로컬 이력 도구이고 GitHub는 저장소 호스팅과 협업 기능을 제공하는 웹 서비스입니다. Pull request는 변경을 합치기 전에 목적·차이·확인을 제안하고 검토하는 요청입니다.

왜 필요한가. 커밋과 전송만으로 협업 검토가 끝나는 것은 아닙니다. 다른 사람이 이해할 설명과 확인 범위를 함께 제시해야 합니다. 계정이 없어도 제공한 변경 제안 꾸러미로 리뷰를 연습할 수 있습니다.

함께 보는 예. data/pr_proposal.md와 pr_diff.txt에는 실행 폴더를 명확히 하는 제안이 있습니다. 학생은 목적·실제 변경·검증 근거를 읽고 review_response.md에 수락 또는 보완 이유를 씁니다.

헛갈리는 경우. PR은 로컬 Git 명령의 커밋과 동일하지 않고, 요청을 만들었다고 자동으로 병합되었다는 뜻도 아닙니다. 이번 활동은 제공 카드의 모의 검토이며 GitHub에 실제 요청을 만들지 않습니다.

제공 입력

PR 제안: README 실행폴더 안내 추가.

변경: Run checker. → Run checker from course.

확인: 안내문구 대조만 수행, 프로그램 새 실행은 미수행.

학생 적용

- 제안의 문제와 바뀐 줄을 연결합니다.
 - 검증했다는 문구가 실제 확인 범위와 맞는지 봅니다.
 - 보완 질문 또는 수락 근거를 작성합니다.
 - 로컬 commit·push·PR·merge 역할을 네 칸으로 나눕니다.
- 예상 결과: 계정 없는 모의 리뷰와 실제 GitHub 작업을 구별한 검토 기록.
- 확인 질문: PR 본문에 ‘모든 테스트 통과’라고 쓰면 그대로 믿어도 되나요?
- 막혔을 때: 화면이 없어서 막히면 자료의 제안·diff·검토 응답을 순서대로 읽습니다. 실제 UI 사용 경험이라고 기록하지 않습니다.
- [활동 자료와 작성칸](#)에서 A15를 엽니다.

A15 · 직접 해보기

A15 · GitHub와 PR의 역할

추가 원문·작업 파일:

- [A15_input.md](#)

- [pr_proposal.md](#)
- [pr_diff.txt](#)
- [A15_notes.md](#)
- [review_response.md](#)

목적: 커밋과 전송만으로 협업 검토가 끝나는 것은 아닙니다. 다른 사람이 이해할 설명과 확인 범위를 함께 제시해야 합니다. 계정이 없어도 제공한 변경 제안 꾸러미로 리뷰를 연습할 수 있습니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A15_input.md](#)
- [A15_notes.md](#)

단계

1. 제안의 문제와 바뀐 줄을 연결합니다.
2. 검증했다는 문구가 실제 확인 범위와 맞는지 봅니다.
3. 보완 질문 또는 수락 근거를 작성합니다.
4. 로컬 commit·push·PR·merge 역할을 네 칸으로 나눕니다.

예상 결과: 계정 없는 모의 리뷰와 실제 GitHub 작업을 구별한 검토 기록.

확인 방법: PR 본문에 '모든 테스트 통과'라고 쓰면 그대로 믿어도 되나요?

오류 해결: 화면이 없어서 막히면 자료의 제안·diff·검토 응답을 순서대로 읽습니다. 실제 UI 사용 경험이라고 기록하지 않습니다.

수정 전 예상: _____

확인한 결과와 이유: _____

함께 보는 예와 더 해보기: 원격 이름과 로컬 공유 저장소 / clone과 새 기록 받아오기

[이 교시의 다른 예와 코드 해설 열기](#)

6교시 · 협업 사례와 인계

16:00~16:50 · 설명15분 / 시연8분 / 직접 수행20분 / 풀이7분

A17 · 새 안내 변경을 스스로 기록하기

개념. 종합 활동은 작은 새 요구를 받아 변경·비교·선택·기록을 이어 수행하는 일입니다. 이미 배운 명령을 재사용하며 목적과 근거를 설명합니다.

왜 필요한가. 이전 예제의 정답 명령을 따라 한 것과 새 문구를 올바른 범위에 반영하는 것을 구별합니다. 한 가지 목적에 맞춰 수정하고, 바뀐 줄과 실행 결과를 적습니다.

함께 보는 예. 새 요구는 README에 ‘Missing values stay unknown.’를 추가해 미측정을 추측하지 않는다는 안내를 보강하는 것입니다. 프로그램 코드의 실제 동작을 바꾸었다고 주장하지 않습니다.

헛갈리는 경우. 문서 변경만 했는데 ‘검사기 버그를 고쳤다’라고 메시지를 쓰면 내용과 기록이 맞지 않습니다. 메시지는 실제 변경을 설명해야 합니다.

전체 명령

```
git status --short
git diff -- README.md
git add README.md
git diff --staged
git commit -m "Explain missing-value policy"
git log --oneline -1
```

각 줄의 뜻

줄	코드	뜻
1	<code>git status --short</code>	기존 변경이 있는지 봅니다.
2	<code>git diff -- README.md</code>	새 안내 한 줄을 비교합니다.
3	<code>git add README.md</code>	그 파일을 선택합니다.
4	<code>git diff --staged</code>	기록할 최종 차이를 봅니다.
5	<code>git commit -m "Explain missing-value policy"</code>	실제 문서 목적을 메시지로 남깁니다.
6	<code>git log --oneline -1</code>	생긴 기록을 확인합니다.

실행 위치와 확인

work/git-lab로 돌아와 상태를 확인합니다. README에 지정 문장을 추가하고 저장합니다. peer에서 작업하지 않도록 경로를 확인합니다.

대표 확인 결과(커밋 해시·사용자명 등은 환경마다 다름):

```
<새해시> Explain missing-value policy
README에 Missing values stay unknown.가 포함됩니다.
```

학생 적용

1. 새 요구와 수정할 파일을 기록합니다.
2. 지정 한 문장만 추가합니다.
3. 비교·선택·재검토·커밋을 수행합니다.
4. 해시·메시지·실제 차이를 인계 노트에 남깁니다.

예상 결과: 새 안내를 담은 목적이 분명한 커밋.

확인 질문: 기록 뒤 공유 저장소에도 새 커밋이 있다고 말해도 되나요?

막혔을 때: 다른 변경이 섞여 있으면 그 목적을 먼저 확인하고 한 파일만 선택했다고 모든 내용이 맞다고 가정하지 않습니다.

[활동 자료와 작성칸](#)에서 A17를 엽니다.

A17 · 직접 해보기

A17 · 새 안내 변경을 스스로 기록하기

목적: 이전 예제의 정답 명령을 따라 한 것과 새 문구를 올바른 범위에 반영하는 것을 구별합니다. 한 가지 목적에 맞춰 수정하고, 바뀐 줄과 실행 결과를 적습니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A17_input.md](#)
- [A17_final_change.txt](#)
- [A17_notes.md](#)
- [A17_final_change.txt](#)

단계

1. 새 요구와 수정할 파일을 기록합니다.
2. 지정 한 문장만 추가합니다.
3. 비교·선택·재검토·커밋을 수행합니다.

4. 해시·메시지·실제 차이를 인계 노트에 남깁니다.

예상 결과: 새 안내를 담은 목적이 분명한 커밋.

확인 방법: 기록 뒤 공유 저장소에도 새 커밋이 있다고 말해도 되나요?

오류 해결: 다른 변경이 섞여 있으면 그 목적을 먼저 확인하고 한 파일만 선택했다고 모든 내용이 맞다고 가정하지 않습니다.

수정 전 예상: _____

확인한 결과와 이유: _____

A18 · 변경 이력과 확인 범위 인계

개념. 인계는 다음 사람이 현재 상태와 변경 이유를 확인하도록 파일·브랜치·커밋·남은 변경·공유 여부를 설명하는 일입니다. 명령 실행 여부와 결과 해석을 함께 적습니다.

왜 필요한가. '완료'만 전달하면 저장만 했는지, 커밋했는지, 공유했는지 구별할 수 없습니다. 수신자가 어느 상태에서 시작할 지 판단할 근거가 필요합니다.

함께 보는 예. change_report.md에 작업 저장소 경로, 최신 메시지와 해시, 수정 문장, 확인한 diff, 로컬 공유 여부, GitHub 미사용을 적습니다. 새 사람은 로그와 파일을 대조합니다.

헛갈리는 경우. 작업 폴더를 ZIP으로 전달할 때 .git 포함 여부에 따라 이력 전달 여부가 달라집니다. 제출 정책에 맞춰 상태 보고를 작성하고 누락된 이력을 있다고 말하지 않습니다.

제공 입력

인계 칸: 저장소 경로 / 브랜치 / 최신커밋 / 변경 목적 / diff 근거 / 남은 변경 / 공유 대상·상태 / GitHub 수행 여부.

학생 적용

1. status-log-remote 결과로 각 칸을 채웁니다.
2. 짝이 같은 커밋과 문장을 찾는지 확인합니다.
3. 로컬 공유와 GitHub 미사용을 구별합니다.
4. 제출물의 이력 포함 여부를 기록합니다.

예상 결과: 저장·기록·공유를 구별한 재현 가능한 인계.

확인 질문: clean 상태이면 원격도 최신이라고 단정할 수 있나요?

막혔을 때: 결과가 불명확하면 추정하지 말고 실제 명령과 출력을 남겨 다음 확인을 요청합니다.

[활동 자료와 작성칸](#)에서 A18를 엽니다.

공식 참고자료

- [git-status](#)

- [git-diff](#)

- [git-add](#)

- [git-commit](#)
- [git-restore](#)
- [git-remote](#)
- [git-push](#)
- [git-pull](#)
- [hello-world](#)
- [git-init](#)
- [git-clone](#)

공식 동작은 2026-09-26 확인했습니다. 자체 합성 사례로 설명하며 현재 교실 환경이나 학습 효과를 실측한 결과는 아닙니다.

개념 보충 공식 근거 (2026-10-05 확인)

- [Git restore: 기준과 작업·선택 영역](#)
- [Git merge: 충돌과 중단](#)

A18 · 직접 해보기

A18 · 변경 이력과 확인 범위 인계

추가 원문·작업 파일:

- [A18_input.md](#)
- [A18_notes.md](#)
- [change_report.md](#)

목적: '완료'만 전달하면 저장만 했는지, 커밋했는지, 공유했는지 구별할 수 없습니다. 수신자가 어느 상태에서 시작할지 판단할 근거가 필요합니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A18_input.md](#)
- [A18_notes.md](#)

단계

1. status·log·remote 결과로 각 칸을 채웁니다.
2. 짝이 같은 커밋과 문장을 찾는지 확인합니다.
3. 로컬 공유와 GitHub 미사용을 구별합니다.

4. 제출물의 이력 포함 여부를 기록합니다.

예상 결과: 저장·기록·공유를 구별한 재현 가능한 인계.

확인 방법: clean 상태이면 원격도 최신이라고 단정할 수 있나요?

오류 해결: 결과가 불명확하면 추정하지 말고 실제 명령과 출력을 남겨 다음 확인을 요청합니다.

수정 전 예상: _____

확인한 결과와 이유: _____

함께 보는 예와 더 해보기: 충돌 표식과 내용 결정

[이 교사의 다른 예와 코드 해설 열기](#)

오늘 배운 내용 확인

확인할 것	배점	충족 기준
배운 내용 설명	20점	사용한 개념과 선택한 이유를 예로 설명한다.
결과물 완성	40점	오늘 필수 활동의 파일·표·기획 내용을 빠짐없이 완성한다.
결과 확인	30점	예상과 결과를 비교하고 다른 입력 한 가지로 다시 확인한다.
저장과 전달	10점	파일 이름과 실행·읽는 방법을 함께 적어 다른 사람이 확인할 수 있다.

각 항목: 모두 충족하면 배점 전부, 일부 충족하면 배점의 절반, 근거가 없으면 0점. 오류를 발견하고 고친 기록도 확인 근거로 사용합니다.

남길 결과: A17_notes.md, A17_final_change.txt / A18_notes.md, change_report.md

세 가지 확인 질문

1. 파일을 저장하는 것과 Git에 커밋하는 것은 어떻게 다른가요?

내 설명: _____

2. 오늘 필수 활동에서 값을 한 곳 바꾼 예를 고르세요. 바꾸기 전 예상과 확인한 결과를 설명해 보세요.

내 설명: _____

3. 마지막 결과물의 파일 이름과 읽거나 실행하는 방법을 적어 보세요. 다른 사람이 확인할 수 있도록 한 문장씩 안내합니다.

내 설명: _____

더 해보기 · 추가 확인 문제

구간별 확인 질문

1. status를 열 번 실행하면 커밋이 생기나요?
2. 커밋 해시가 교안과 달라지면 실패인가요?
3. .gitignore가 과거 커밋의 파일을 지우나요?
4. 브랜치를 만들면 GitHub PR도 자동으로 생기나요?
5. PR 본문에 '모든 테스트 통과'라고 쓰면 그대로 믿어도 되나요?
6. clean 상태이면 원격도 최신이라고 단정할 수 있나요?